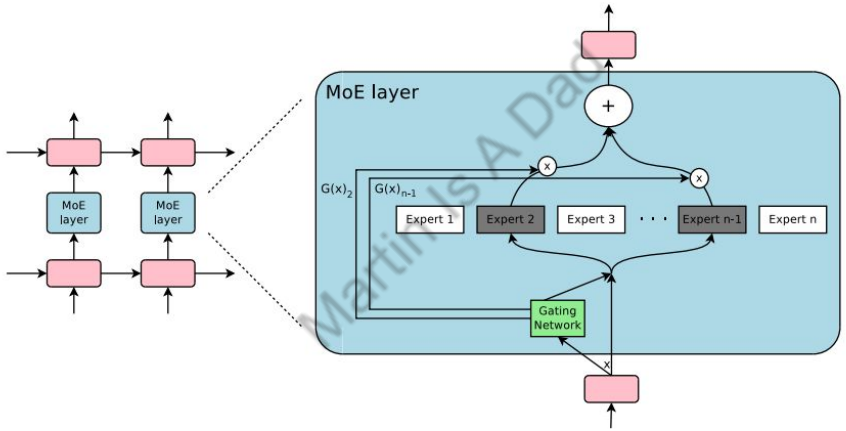
Mixture Of Experts

Mixture of Experts

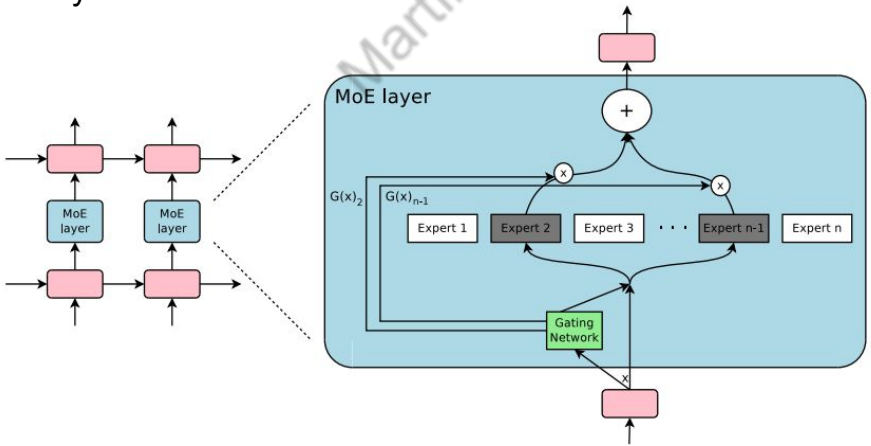
- MoE is an architecture where the computation for each input is handled by a combination of multiple specialized sub-networks, called "experts." Instead of using a single large, dense network for all inputs, MoE selectively activates only a subset of these experts based on the input. Originally came out earlier than the famous Transformer paper in 2017, also from Google



- Key Components:
  - **Experts**: Individual neural networks (often smaller LLMs or feed-forward networks) that are trained to specialize in different tasks.
  - **Router** (or Gating Network): Separate neural network that analyzes the incoming input and decides which experts are most relevant to process it. Essentially acts as a dispatcher.
  - **Combination Mechanism**: The outputs from the selected experts are then combined (e.g., through weighted averaging or summation) to produce the final output of the MoE layer.

Mixture of Experts Typical Flow

- Input fed into an LLM with an MoE layer (usually replacing Transformer's FFN layer), the router network processes this input.
- Based on the input, the router assigns a weight or score to each expert, indicating its relevance to the current input.
- Typically, only the top-k experts (where 'k' is a predefined number, often a small value like 1 or 2) with the highest scores are activated for that specific input. This is known as sparse MoE, which is more efficient. There are also dense MoE models where all experts contribute to the output, but with varying weights.
- The activated experts independently process the input.
- Their outputs are then combined according to the weights assigned by the router to produce the final output of the MoE layer.



Mixture of Experts Benefits

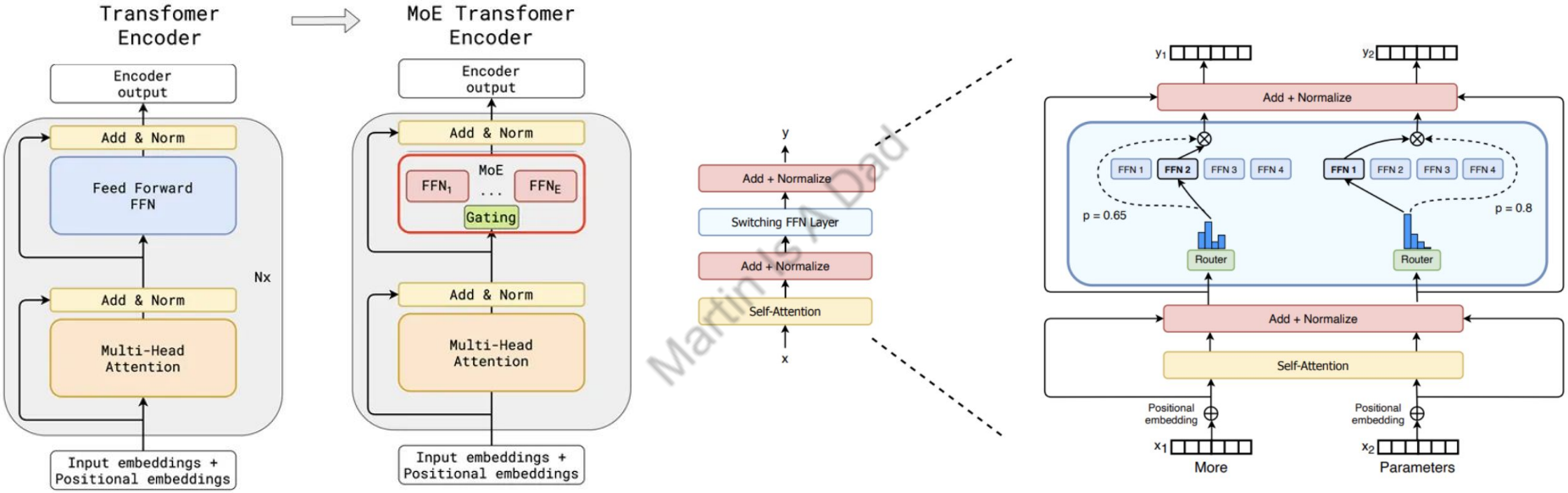
- **Increased Model Capacity:** MoE allows for creating models with a significantly larger total number of parameters compared to dense models with similar computational cost. Each expert contributes its own set of parameters, leading to a higher overall capacity to learn complex patterns.

- **Improved Efficiency:** During inference, only a small fraction of the total parameters (those of the selected experts) are actually used for each input. This leads to faster inference speeds and lower computational costs compared to running a dense model with the same number of parameters.

- **Specialization and Better Performance:** By having experts specialize in different aspects of the data, the model can potentially achieve better performance on a wider range of tasks and handle diverse types of input more effectively. Each expert can become highly proficient in its area of specialization.

- **Faster Pre-training:** Due to the sparse activation of experts, MoE models can be pre-trained more efficiently than dense models with the same capacity.

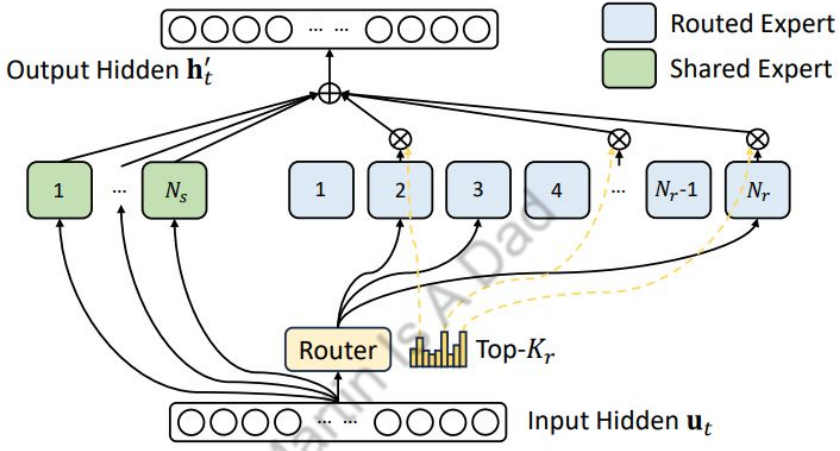
Mixture of Experts Architecture



Images from huggingface.co/blog/moe

Replace every FFN layer of the transformer model with an MoE layer, which is composed of a gate network and a certain number of experts.

DeepSeekMoE (Introduced in DeepSeek-V2)

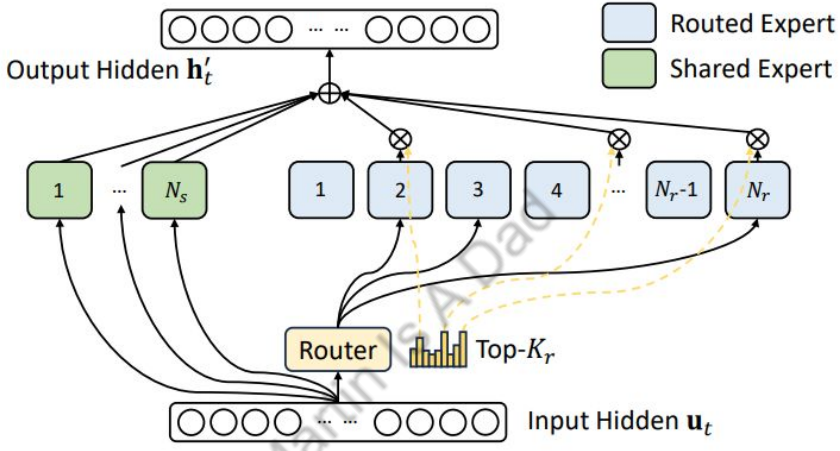


Differences with conventional MoE:

- With small number of experts, different tokens can easily routed to single expert, creating load imbalance. To solve this, Instead of having a smaller number of large experts, DeepSeekMoE finely segments the expert layer into a larger number of smaller, more granular experts. (N_r = 256, K_r = 8)

- Tokens assigned to different experts could require common knowledge. To solve this, V2 isolates a set of experts specifically as "shared experts." These shared experts are designed to capture common, general knowledge that is relevant across various inputs. (N_s=1)

DeepSeekMoE (Introduced in DeepSeek-V2)



Benefits:

- Higher Degree of Specialization: By using more, smaller experts and encouraging them to focus on narrower domains of knowledge.
- Improved Parameter Efficiency: By promoting specialization, the model can potentially achieve better performance with a similar number of active parameters compared to regular MoE models.
- Reduced Redundancy: By having dedicated shared experts handle common knowledge, the routed experts can focus on unique specializations.

MoE Load Balancing

- **Preventing Route Collapse**: Without proper load balancing, a small subset of experts might receive a disproportionately large number of input tokens, while others remain underutilized, also known as "route collapse". This hurts the learning of the underloaded experts, leading to poor overall model performance and generalization.

- **Maximizing Efficiency:** Good load balancing ensures that all experts contribute meaningfully to the model's output. By dynamically distributing tasks, it prevents any single expert from being overwhelmed, which could lead to computational bottlenecks and reduced throughput.

- **Improving Training Stability:** Load imbalance can introduce instability during the training process. When some experts are heavily loaded and others are idle, the learning signals and updates to the model parameters can become skewed, making convergence more difficult.

- **Enhancing Generalization:** If some experts are consistently underutilized, they don't receive enough training data to learn meaningful specializations. This lack of specialization across the expert pool can lead to a model that doesn't generalize well to diverse inputs.

- **Optimizing Parallel Training:** In distributed training scenarios where experts reside on different devices (e.g., GPUs), which is the case for DeepSeek, load imbalance can severely hinder parallel efficiency. If some devices are overloaded while others are idle, the overall training time increases.

Common Load Balancing Solution

- Introduce auxiliary loss for load balancing to original training objective, similar mechanism of doing regularization. Example loss function can be like:

$$\mathcal{L}_{\text{Balance}} = \alpha \sum_{i=1}^{N} f_i P_i,$$

$$f_i = \frac{N}{KT} \sum_{t=1}^{T} \mathbb{1}(\text{ Token } t \text{ selects Expert } i),$$

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s_{i,t},$$

$$s_{i,t} = \text{Softmax}_i\left(\mathbf{u}_t^{l^T} \mathbf{e}_i^l\right),$$

- *N* is the number of experts, *T* is the number of tokens, *K* is the number of activated experts for each input token.
- $s_{i,t}$ is the gating function output, normalized to [0, 1] via Softmax. It means the probability of t-th token choosing i-th expert, also the `affinity score` between *i*-th expert and *t*-th token.
- P_i is the average probability of choosing the *i*-th expert for the entire input sequence, also the average gating score
- f_i represents the fraction of tokens routed to the *i*-th expert
- α is a hyper-parameter controlling the strength of the auxiliary loss

Common Load Balancing Solution

- DeepSeek-V2 is using similar approach. It introduced different levels of load balancing loss:
  - **Expert-Level balance loss:** encourages uniform token distribution **across experts**
  - **Device-Level balance loss:** encourages balanced computation **across devices (GPU)**
  - Drop tokens beyond expert capacity

- However, load balancing auxiliary loss is not free, since its gradient can interfere with LLM's quality objective (just like all over regularizations), leading to worse model performance. A small auxiliary loss coefficient causes insufficient load balancing, whereas a larger value impairs the model performance. A higher MaxVio value indicates a greater degree of imbalance.

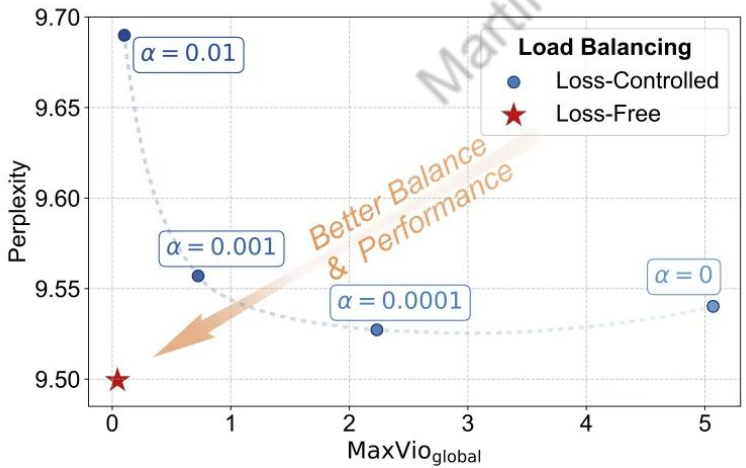
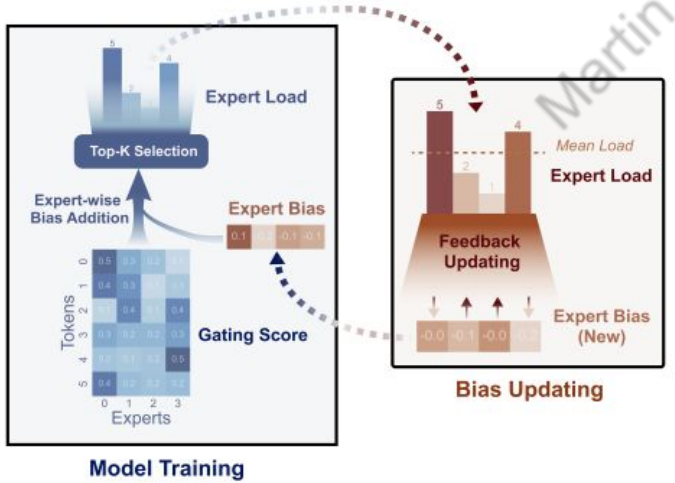


Image from paper [AUXILIARY-LOSS-FREE LOAD BALANCING STRATEGY FOR MIXTURE-OF-EXPERTS]

DeepSeek-V3 Auxiliary Loss-Free Strategy

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j | 1 \leqslant j \leqslant N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases}$$

- Additional bias added for every expert in MoE router, to the corresponding affinity scores $s_{i,t}$ to determine top-K routing (only used for routing and not others like gating scores)
  - The bias values are updated at the end of each training step
  - Increase bias for experts that received less than average number of tokens
  - Decrease it for experts that received more tokens
- No tokens getting dropped due to the effective load balancing strategy



Since Loss-Free Balancing does not produce any interference gradients, it also elevates the upper bound of model performance gained from MoE training.

Image from paper AUXILIARY-LOSS-FREE LOAD BALANCING STRATEGY FOR MIXTURE-OF-EXPERTS

Multi-head Latent Attention

What is KV Cache

- KV Cache is eligible when new token's attention calculation only depend on itself and previous tokens. This is usually true for generative models nowadays (Decoder only GPT family for example).
- Masked attention module will make sure each token in a sequence only attends to previous tokens and itself, not future tokens.
- **(+)**: Number of FLOPs (floating point operation per second) drop Drops from O(t2) to O(t), t is number of sequence
- **(-)**: Extra memory needed. The memory amount is of size [2, num_of_tokens, num_layers, num_kv_heads, kv_dim], 2 as in key and value

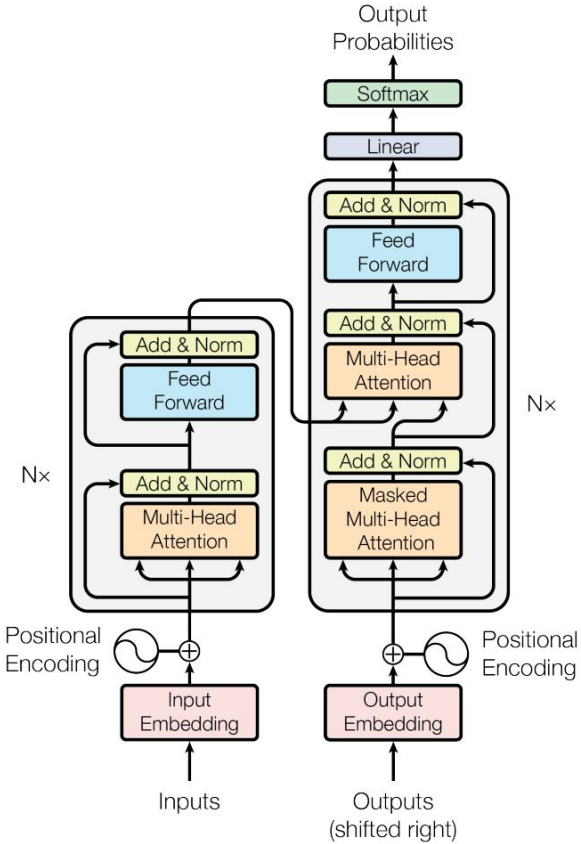
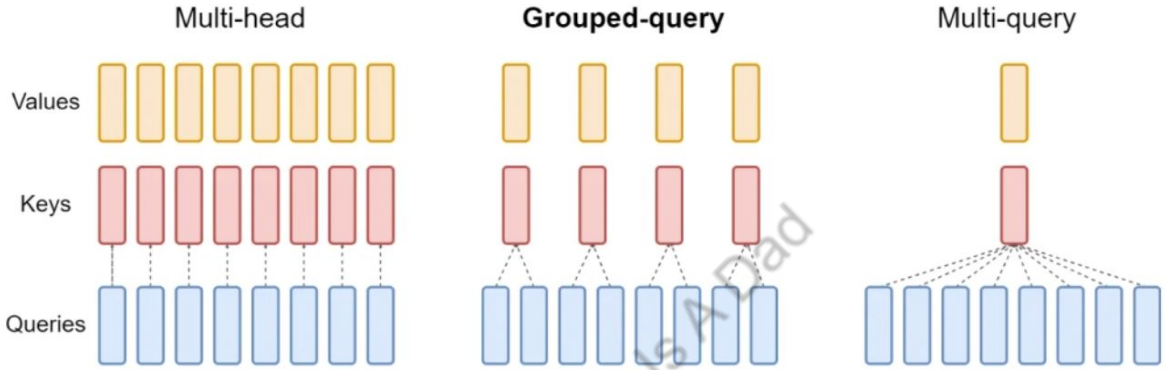


Figure 1: The Transformer - model architecture.

What is Grouped Query Attention?



- KV Cache memory size becomes bottleneck, especially when the sequence is longer and longer. MQA &GQA is to address the concern.
- MQA is to share a single key and a single value head across all query heads, which can significantly reduce memory usage but will also impact the accuracy of attention.
- GQA can be seen as an interpolating method between MHA and MQA, where a single pair of key and value heads will be shared only by a group of query heads, not all queries. The quality impact is less.

GQA Pros

- **Quality** — GQA has quality close to multi-head attention (MHA) by interpolating between multi-query attention (MQA) and MHA, finding a better balance between the two.
- **Speed** — GQA maintains a speed comparable to MQA, which is faster than MHA, by reducing the KV Cache memory consumption compared with MHA.
- **Reduced Computational Complexity** — Similar to speed, GQA can significantly reduce the computational complexity of large language models, leading to faster inference times.
- **Low Memory Usage** — GQA find a better balance at the low memory usage of MQA with the quality of MHA, making it suitable for large-scale models with memory constraints.

GQA Cons

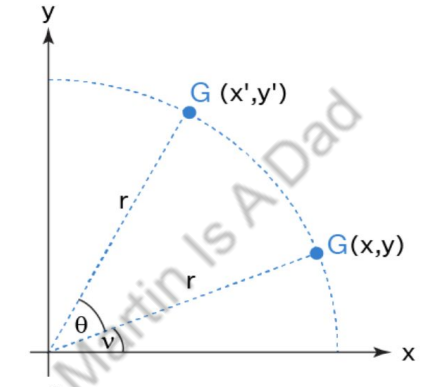
- **Quality Degradation** — GQA attempts to find the perfect balance by using an intermediate number of key-value heads (more than one but fewer than the query heads), but the balance between speed and quality is a challenge.
- **Group Division** — The input nodes are divided into several groups and attention is calculated only within that local block. This division and management of groups add to the complexity of the GQA implementation.
- **Hyperparameter Tuning** — Achieving optimal performance with GQA requires careful tuning of hyperparameters. For instance, the number of groups into which the query heads are divided can significantly impact the model's performance and efficiency.
- **Complexity of Implementation** — Implementing GQA within the context of an autoregressive decoder using a Transformer model can be complex.

RoPE (Rotary Positional Encoding)

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

Absolute Position Encoding

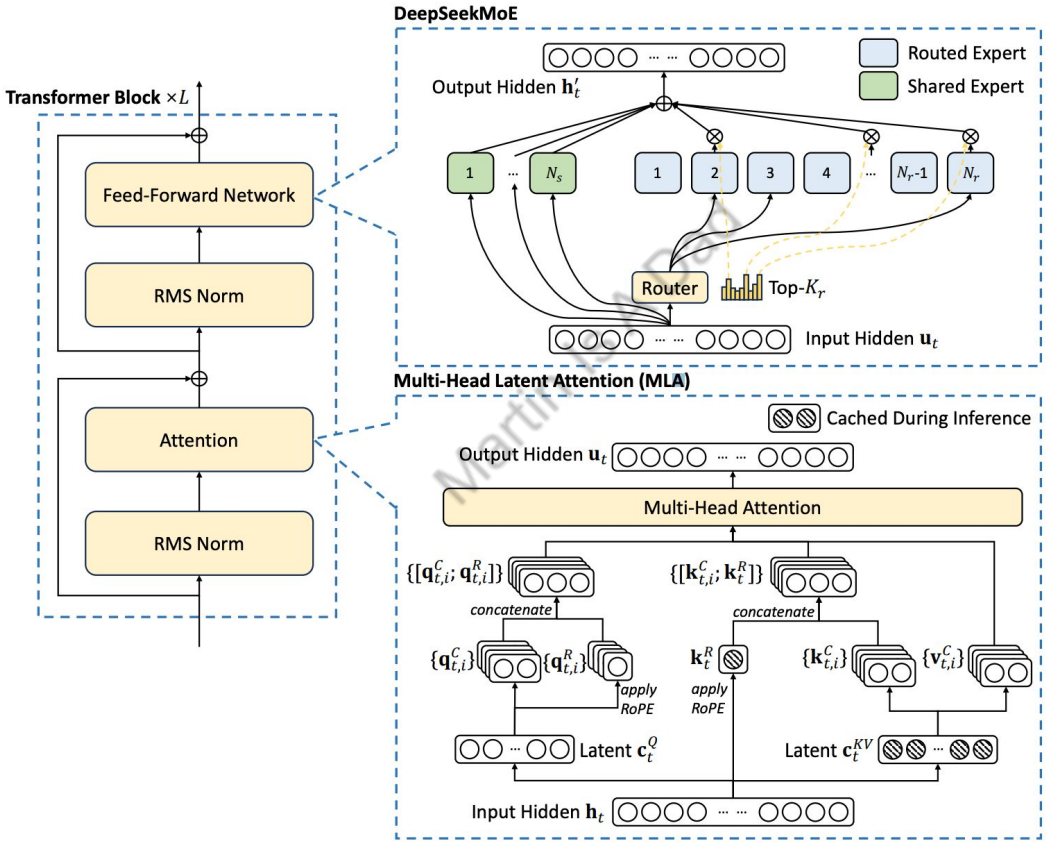


$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}.$$

$$RoPE\left(x_m^{(1)}, x_m^{(2)}, m\right) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix} = \begin{pmatrix} x_m^{(1)} \cos m\theta - x_m^{(2)} \sin m\theta \\ x_m^{(2)} \cos m\theta + x_m^{(1)} \sin m\theta \end{pmatrix}$$

Rotary Positional Encoding

Deepseek's MultiHead Latent Attention (MLA)

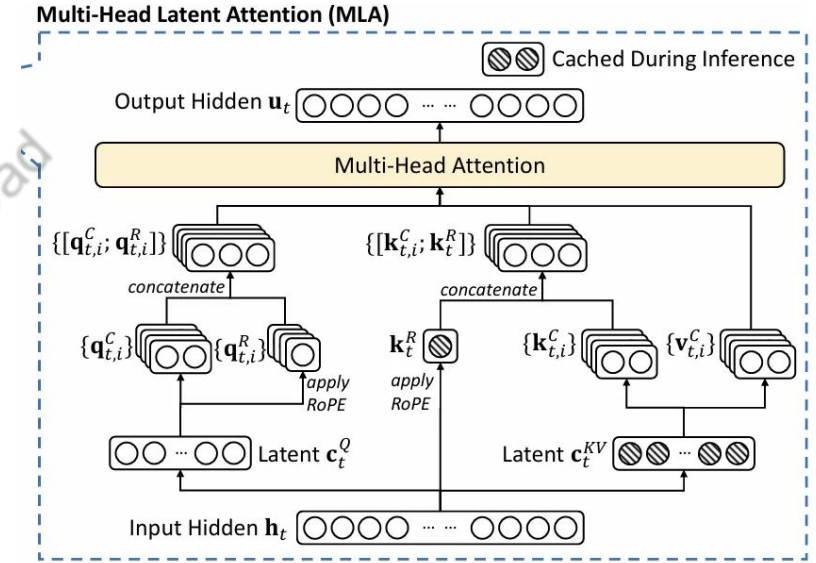


Deepseek's MultiHead Latent Attention (MLA)

- DeepSeek uses a low rank attention mechanism they call "Multi-Head Latent Attention" which projects the keys and values into a shared low-rank space. This equates to reducing the sum of the dimensions of the query, key, and value vectors to below the hidden dimension.

- Since RoPE isn't preserved by the low rank compression (i.e. the RoPE transformation does not commute with the dimension compression matrices), DeepSeek has a single query/key pair that doesn't have a low rank compression matrix. These queries/keys are added by direct sum to the decompressed query/key pairs without position embedding.

- RoPE could be combined with MLA directly with some work. $W^{U*}=\mathscr{W}^{U*}\otimes \boldsymbol{I}_{2\times 2}$, where $\mathscr{W}^{U*}$ has half the dimensionality of the compression matrix W^{U*}. This would ensure that RoPE matrices commute with the low rank compression matrices since the RoPE matrices are composed of 2×2 blocks on the diagonal.



Multi-Token Prediction

Next Token Prediction

	Encoder Input	Decoder Input	Output
Step 1	I am Martin	Start	我
Step 2		我	是
Step 3		我是	马
Step 4		我是马	丁

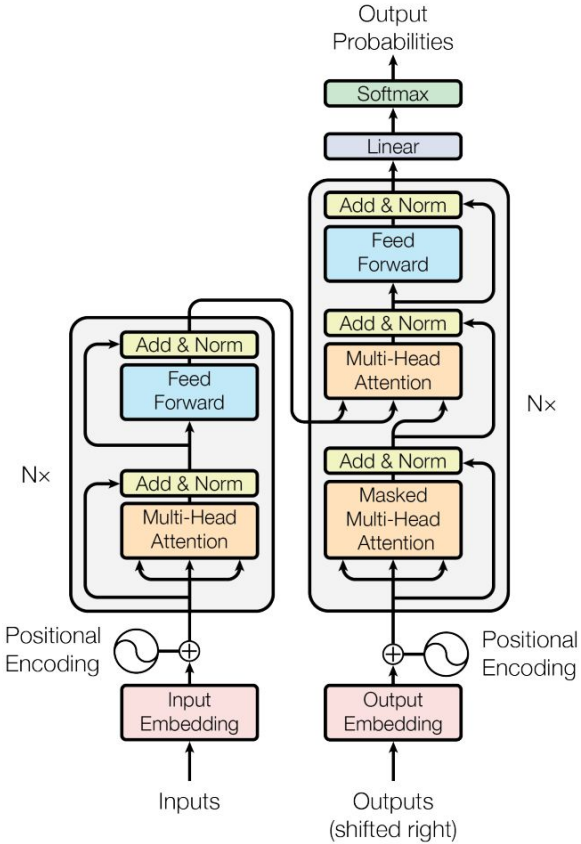


Figure 1: The Transformer - model architecture.

Next Token Prediction Training

- Same concept with other ML techniques: minimize loss function + backpropagation to update weights
- What is the loss function?
  - **Cross Entropy Loss:** a metric used in machine learning to measure how well a model performs by comparing the probability distribution predicted by the model to the actual distribution of the target labels
  - Formula:

$$\text{Loss} = -\sum_t P_{\theta}(x_{t+1} \mid x_{t:1})$$

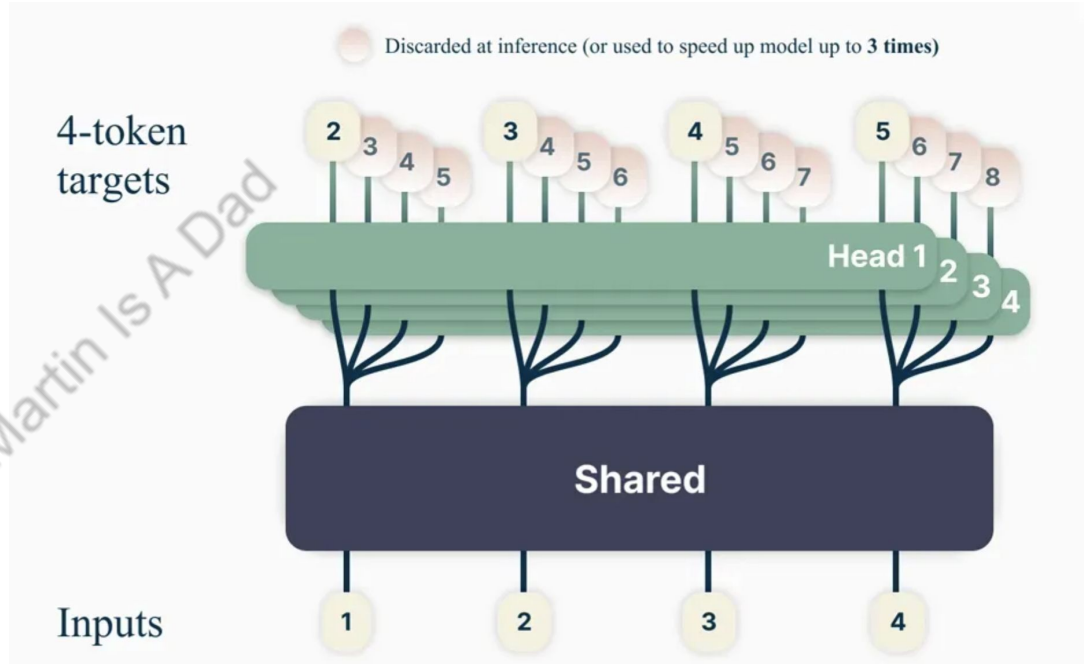
  - Given a sequence $x_1, x_2, \ldots, x_t$ (say [I, am, Ruthie's]) the probability of the model predicting next token to be x_{t+1} is $P_{\theta}(x_{t+1} \mid x_{t:1})$. We are trying to maximize the probability.
- **Is there a more efficient way given same training data?**

Multi-Token Prediction

- The model predict several future words simultaneously instead of just one. At each position in a sentence, the model uses multiple “heads,” to forecast the next several words all at once, working collaboratively to improve efficiency and coherence.

- Loss Formula:

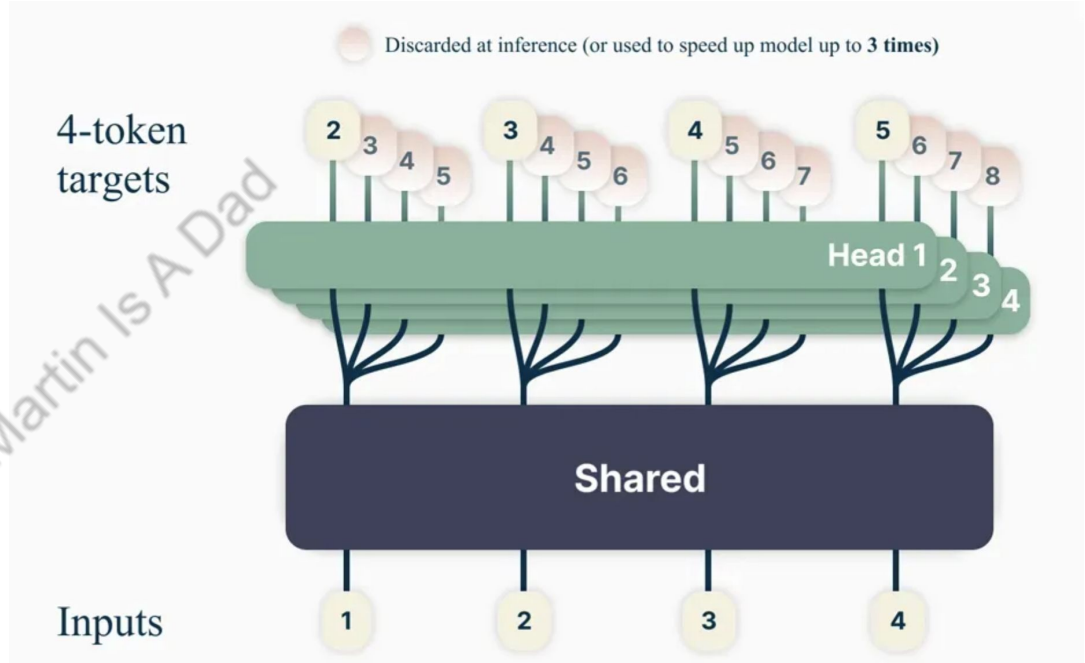
$$\text{Loss} = -\sum_t P_\theta(x_{t+n:t+1} \mid x_{t:1})$$



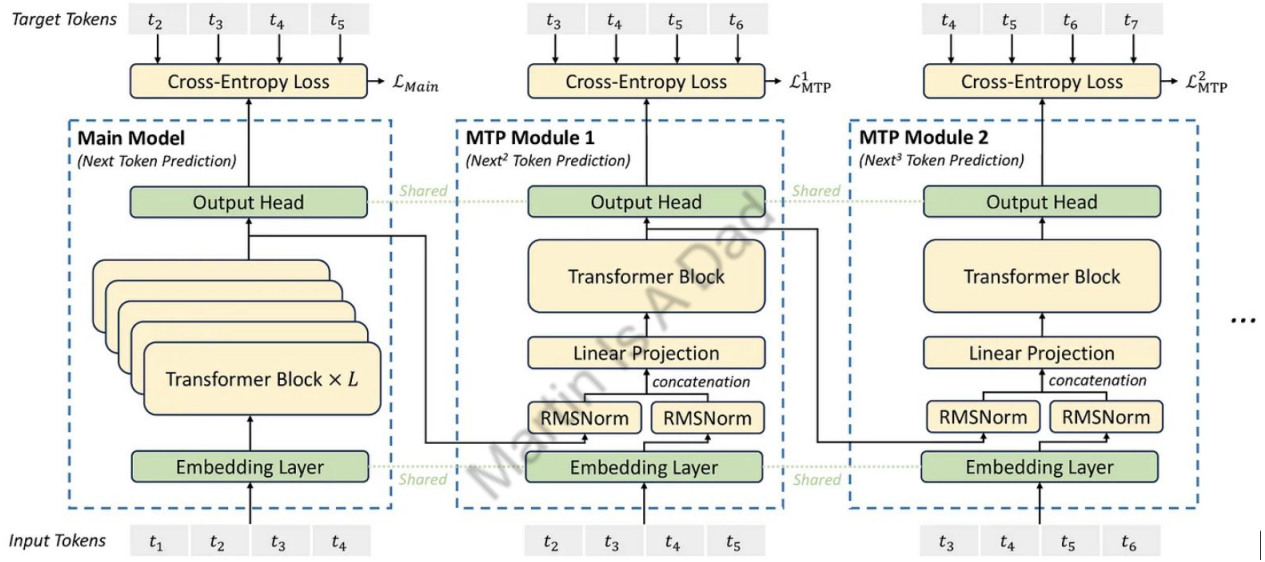
Multi-Token Prediction

Advantages:

- More efficient learning. Multi-token prediction enables the model to learn more effectively from the same dataset.
- Better quality: models capture and predict long-term patterns more accurately, has better quality in tasks requiring content generation, such as code generation for SWE (LAYOFFS~~~~)
- Faster inference (not used in lots of models, probably due to extra model complexity)



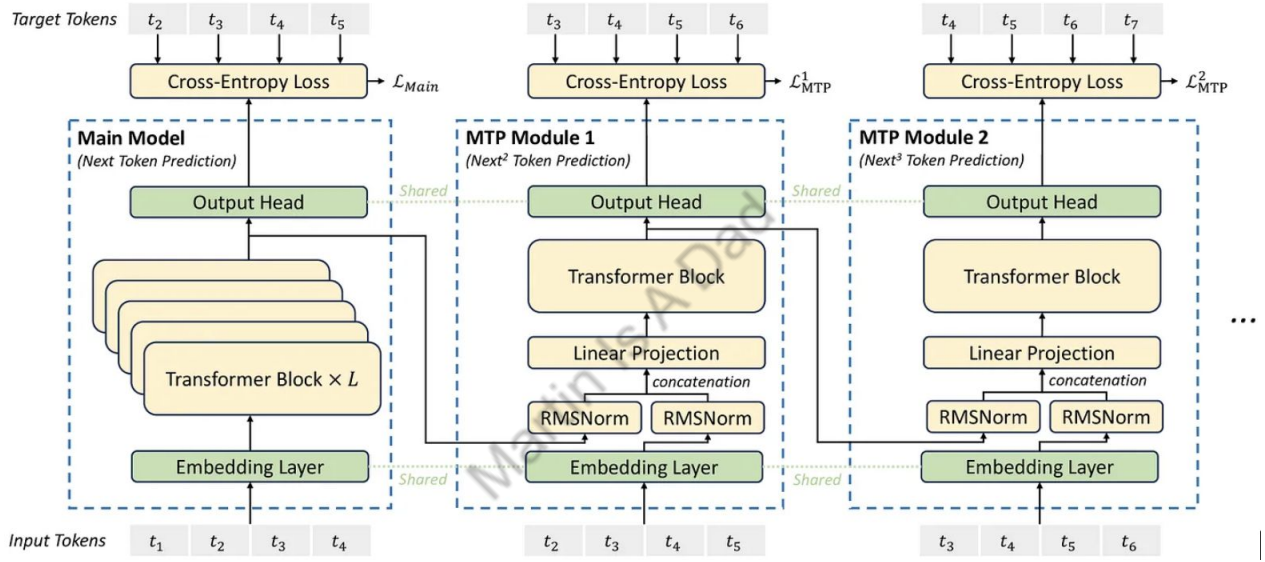
DeepSeek Multi-Token Prediction



Differences:

- Sequential instead of parallel, this it to maintain full causal chain (for example [t_1, t_2, t_3] -> [t_4, t_5], t_5 is generated without context from t_4
- Only used to optimize training, MTP modules are discarded during inference

DeepSeek Multi-Token Prediction



MTP module:

- A **shared** embedding layer: turns tokens into word vectors
- A **shared** output head: linearly transformation of transformer block output and applies the softmax to compute the prediction probabilities
- A Transformer block: attention!
- A projection matrix: linear transformation
- Concat RMS Norm: stabilize the training process

Mixed Precision Training And More

What is FP16

- FP16 refers to 16 bit floating point. Normally programs run with 32 bit or 64 bit numbers. By using FP16 to represent model weights and activations, it allows faster computation and significantly reduce memory usage

- Advantages
  - **Smaller Memory Usage:** FP16 need 50% of memory required compared to float32, enabling larger batch sizes and models to fit in the GPU memory.
  - **Faster Computation:** Many GPUs are optimized for FP16, allowing accelerated matrix MATMUL without significant degradation in accuracy.
  - **Acceptable Precision:** FP16 retains enough dynamic range and numerical stability with proper handling of quantization (the process of mapping continuous infinite values to a smaller set of discrete finite values) underflows and overflows

So can be do better than FP16, say FP8?

What is FP8

- FP8 refers to 8 bit floating point.

- Advantages (In addition to FP16)
  - **Even smaller Memory Usage:** FP8 need 50% of memory required compared to FP16, enabling larger batch sizes and models to fit in the GPU memory.
  - **Higher Throughput:** Training with FP8 enables faster computation, smaller data types reduce data transfer time and allow for more FLOPs (floating point operations per second) on hardware optimized for lower precision.
  - **Lower Energy Consumption:** Reducing precision can drastically decrease energy costs, a crucial factor in sustainable AI development.

However, the transition from FP16 to FP8 is tricky. FP8 has several unique challenges compared with FP16, which has established a stable environment for most models and hardwares.

FP8 Challenges: Numerical Instability

- **Gradient Underflow:** During backpropagation, gradients can become extremely **small**. FP8's reduced range struggles to represent these values accurately, leading to unstable or non-converging training. A value during the process of quantization (converting continuous values to discrete ones), becomes smaller than the smallest representable value allowed by the data type, essentially resulting in the value being rounded down to zero due to the limitations of precision. Small values critical for fine-tuned updates may be rounded to zero, affecting the model's ability to learn subtle patterns.

- **Gradient Overflow:** During backpropagation, gradients can become extremely **large**. FP8's reduced range struggles to represent these values accurately, leading to unstable or non-converging training. A value being quantized (converted to a lower precision representation) exceeds the maximum representable value within the chosen data type, essentially "overflowing" the allowed range and resulting in an inaccurate representation due to the limitations of the quantization process; it's a type of error that occurs when a value becomes too large to be accurately represented within the specified bit-depth during quantization.

DeepSeek: Blockwise Quantization Scale

- To prevent overflow, we typically scales a higher-precision weight or activation matrix down to the FP8 representable range before quantizing it. For example, by dividing the whole tensor by its maximum element. That maximum element is kept separately and used as a scaling factor in each MATMUL (or GEMM).

- However, this makes the quantization process extremely sensitive to outliers: the presence of a very large weight in some layer could force all other weights to be quantized to 0. DeepSeek team handles this issue by introducing **blockwise** and **tilewise** scaling, in which each 128×128 submatrix of a weight matrix, respectively each 1×128 subvector of an activation vector, is scaled and quantized separately.

FP8 Challenges: Error Accumulation

Errors introduced due to FP8's lower precision can propagate and accumulate through layers of the model, compounding over time and seriously breaking the quality of the model. This is even worse for large models with many layers, like Transformers, where errors accumulate across thousands of layers.

DeepSeek: Mixed Precision Training

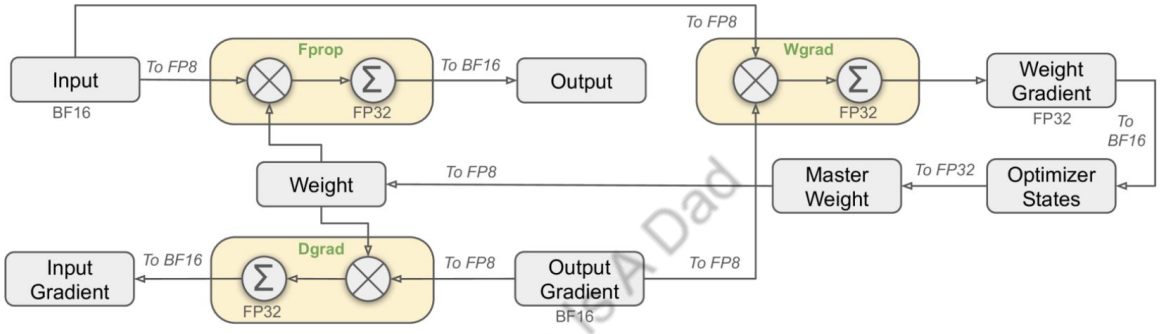


Figure 6: The overall mixed precision framework with FP8 data format. For clarification, only the `Linear` operator is illustrated.

- **FP32 accumulation:** Matrix multiplications are performed in FP8, the accumulation of results is done in FP32 for higher precision.
- **Hybrid precision:** Model weights are stored in FP8, activations and gradients are stored in BF16. This balance helps preserve important information during training. FP32 is used for some internal computation.

FP8 Challenges: Hardware Limitation

- Hardwares like NVDIA Hopper architecture are starting to support more and more of FP8, but there’re still lots of gaps for optimal performance.

- Low-precision MATMUL operations often suffer from underflow issues, and their accuracy largely depends on high-precision accumulation, which is commonly performed in an FP32 precision. However, DeepSeek observe that the accumulation precision of FP8 MATMUL on NVIDIA H800 GPUs is limited to retaining around 14 bits, which is significantly lower than FP32 accumulation precision. This problem will worsen when the inner dimension K is large, a typical scenario in large-scale model training where the batch size and model width are increased.

DeepSeek: Tensor Core + CUDA Core

- To overcome the hardware limitation, DeepSeek's solution was to move some of the accumulation outside the Tensor Cores.

- Their GEMM (or MATMUL) kernel performs each series of 4 consecutive WGMMA (asynchronous warpgroup-level matrix multiply and accumulate operation) operations inside the Tensor Cores, accumulating in the lower-precision format, but then adds the result into a separate register-backed accumulator tensor in FP32. This second addition is performed using CUDA Cores and thus takes place in ordinary FP32 precision, mitigating the loss of accuracy.

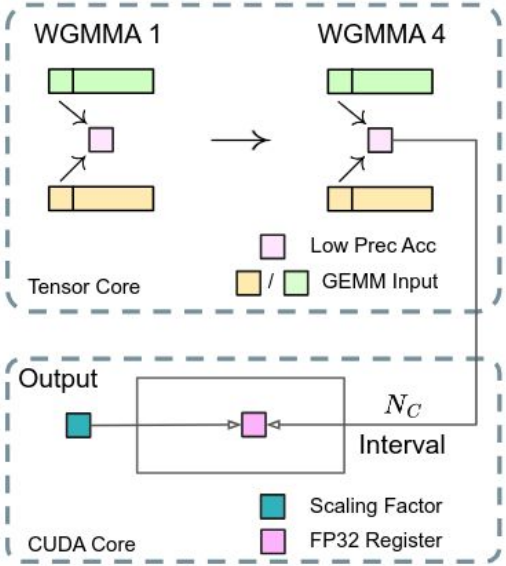


Figure 7(b) of the paper: the mixed-precision matmul technique used during training of DeepSeek-V3, in which lower-precision WGMMA operations on Tensor Cores alternate with higher-precision accumulation in CUDA Cores.